Deadlocks - Unit 2 (Rohan's notes)

Deadlock: a set of processes each waiting for a resource held by another process in the set.

Four necessary conditions (Coffman):
1. Mutual exclusion 2. Hold and wait 3. No preemption 4. Circular wait
All four must hold at the same time for a deadlock to happen.

Resource allocation graph: a cycle means a possible deadlock; with single-instance resources a cycle means a deadlock for sure.

DEADLOCK PREVENTION - make sure at least one condition can never hold:
- Mutual exclusion: use sharable resources where possible (read-only files).
- Hold and wait: a process must request all resources at once before it starts.
- No preemption: if a request cannot be granted, release everything held.
- Circular wait: number all resource types, request only in increasing order.

Deadlocks - Unit 2 (contd.)

DEADLOCK AVOIDANCE - the OS checks every request and grants it only if the system stays in a safe state.
Safe state: there is at least one order (safe sequence) in which every process can finish.
Unsafe state does not always mean deadlock, but it may lead to one.

BANKER'S ALGORITHM (multiple instances of each resource type)
Data structures: Available, Max, Allocation, Need = Max - Allocation.
Safety algorithm: find a process whose Need <= Work, pretend it finishes, add its Allocation to Work, repeat.
Resource-request algorithm: if Request <= Need and Request <= Available, pretend to allocate and run the safety algorithm; grant only if the new state is safe.

Example: 5 processes P0-P4, resources A B C with Available = (3, 3, 2).
Safe sequence found: P1, P3, P4, P0, P2 -> the system is in a safe state.

(see the lecture slides for the rest)